DATA 260 Homework 8

Name: Akanksha Shukla

GitHub repository: https://github.com/akankshashuklaWork/data260-7838

Personal configuration

Value	Result	How it was calculated
SID4	7838	Last four digits of my SJSU Student ID
PORT_BASE	8638	8000 + (7838 mod 900) = 8000 + 638
PREFIX	s7838	"s" + SID4
SEED	7838	SID4
VERIFY_SEED	267838	260000 + SID4
DOMAIN_ID	6	7838 mod 8 = 6, which is Rental Housing Listings
Hardware	MacBook Pro	Apple M4 Pro chip, 48 GB memory
Local model	qwen2:7b	Run locally through Ollama
Tagged commit hash	28cfe7f05d68d0ac9bc63af3c1d1e0a33f6bc976	Commit tagged hw8

Words you need to know first

These are the building blocks. Every later section uses them, so it helps to read this list once.

- **Kafka:** a program that stores and delivers messages. One program drops a message in, another program picks it up later. The two programs never talk to each other directly.
- **Topic:** a named mailbox inside Kafka. Messages are written to a topic and read from a topic. Mine all start with my prefix, for example s7838_tasks.
- **Producer:** any program that writes (sends) messages to a topic.
- **Consumer:** any program that reads messages from a topic.
- **Partition:** a topic can be split into lanes called partitions. Kafka keeps messages in order inside one lane, but it does not promise any order across different lanes.
- **Offset:** the position number of a message inside a partition (0, 1, 2, ...). A consumer remembers how far it has read by saving an offset.
- **Commit (an offset):** telling Kafka 'I finished everything up to here.' If a consumer restarts, it continues from the last committed offset.
- **Consumer group:** a team of consumers that share the work of one topic. Each partition is read by only one member of the team at a time.
- **correlation_id:** a unique label (a UUID) that I stick on a request. Every reply carries the same label, so I can tell which reply belongs to which request.
- **Worker:** the background program that does the real work (here: creating and finding listings).
- **FastAPI:** the Python web framework I use for the small HTTP wrapper in front of Kafka.
- **Docker / container:** a way to run a program in a small sealed box so it behaves the same on any computer.
- **Idempotent:** doing the same message twice has the same effect as doing it once.
- **Poison message:** a message that always fails, no matter how many times it is retried.

- **Dead-letter topic:** a 'parking lot' topic where messages that keep failing are moved so they stop blocking others.

Part 1 - Backend request/response over Kafka

What I built

- In earlier homework, the web app did everything inside one program. In HW8 I split the work into a **client side** and a **worker side** that only talk through Kafka.
- The client side is a FastAPI wrapper. When someone calls it, it does not create a listing itself. It writes a request message to Kafka and waits for the answer.
- The worker is a separate program running in its own Docker container. It reads the request, does the work, and writes the answer back to Kafka.
- Why do this? Splitting the program makes problems visible that a single program hides: lost, duplicated, or out-of-order messages. Part 3 tests exactly those.
- My primary entity is a **rental housing listing**. So the assignment's CREATE_ENTITY becomes **CREATE_LISTING**, GET_ENTITY becomes **GET_LISTING**, and entity_id becomes **listing_id**.

The two topics

- **s7838_entity_requests:** the client/API writes requests here. The worker reads from here.
- **s7838_entity_responses:** the worker writes replies here. The API reads from here.
- The topic names keep the word 'entity' exactly as written in the assignment, even though my operations say LISTING.
- Both topics have 1 partition. This matters because the worker keeps listings in an in-memory Python dictionary, so only one worker should handle them.

Request and response message formats

```
# Request (API -> s7838_entity_requests)
{"correlation_id": "uuid", "reply_to": "s7838_entity_responses",
 "operation": "CREATE_LISTING",
 "data": {"propertyTitle": "...", "propertyLocation": "...", "submitterEmail": "...",
          "propertyDescription": "(more than 25 characters)",
          "propertyCategory": "Apartment|House|Condo|Townhouse", "termsAccepted": "true"}}

{"correlation_id": "uuid", "reply_to": "s7838_entity_responses",
 "operation": "GET_LISTING", "data": {"listing_id": "uuid"}}

# Response (worker -> the topic named in reply_to)
{"correlation_id": "same uuid", "success": true, "listing_id": "uuid", "message": "Listing created
    "}
{"correlation_id": "same uuid", "success": true, "listing": {"listing_id": "uuid", ...fields}}
{"correlation_id": "same uuid", "success": false, "error": "Listing not found"}
```

- **correlation_id:** the label that ties a reply to its request.
- **reply_to:** tells the worker which topic to answer on.
- **operation:** what to do (CREATE_LISTING or GET_LISTING).
- **data:** the details the operation needs.
- **success:** true or false in every reply. A failed reply carries a short, clean error message instead of a crash.

How the correlation_id travels (step by step)

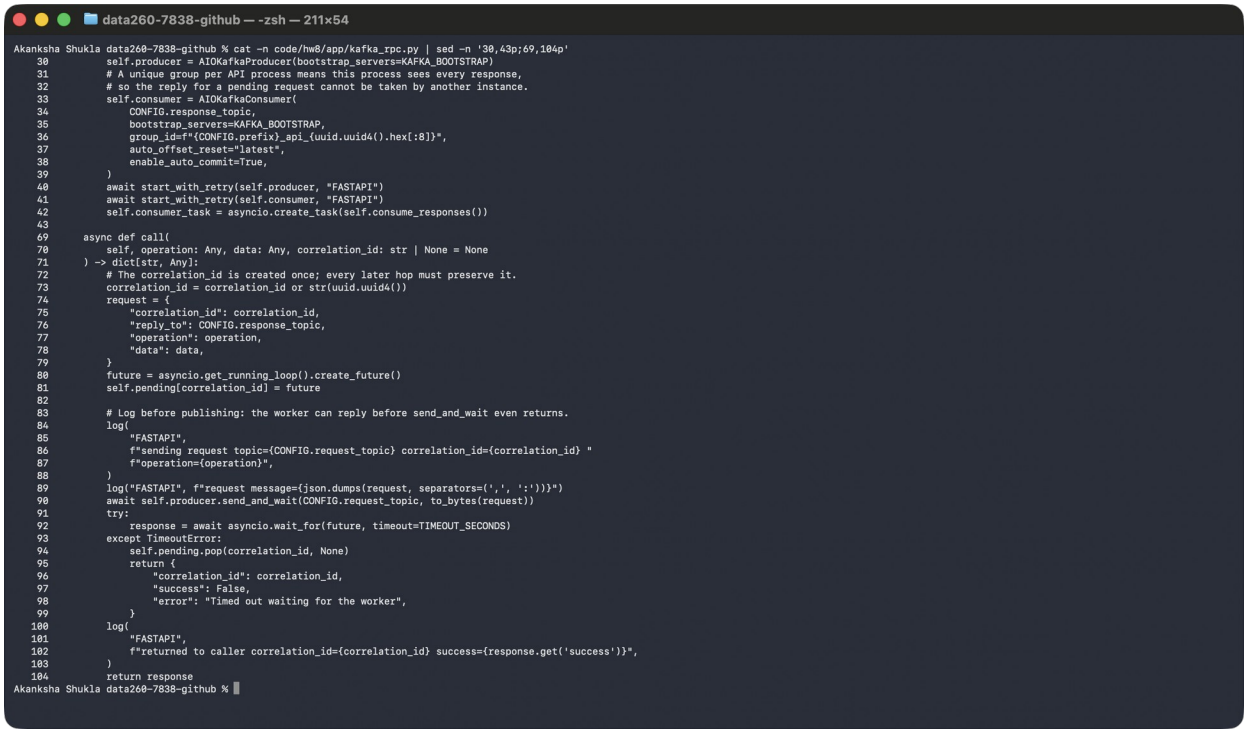
1. The client (homework_demo.py) creates ONE correlation_id before it sends anything.
2. It puts that id in the HTTP request to FastAPI.

3. FastAPI copies the id into the Kafka request message and remembers 'I am waiting for a reply with this id'.
4. The worker reads the request and copies the same id, unchanged, into its reply. Error replies keep it too.
5. FastAPI reads the reply, finds the waiting request by id, and hands the answer back to the caller.
6. The demo checks that the id in the reply equals the id it created. The result printed is: correlation_id preserved end to end: True.

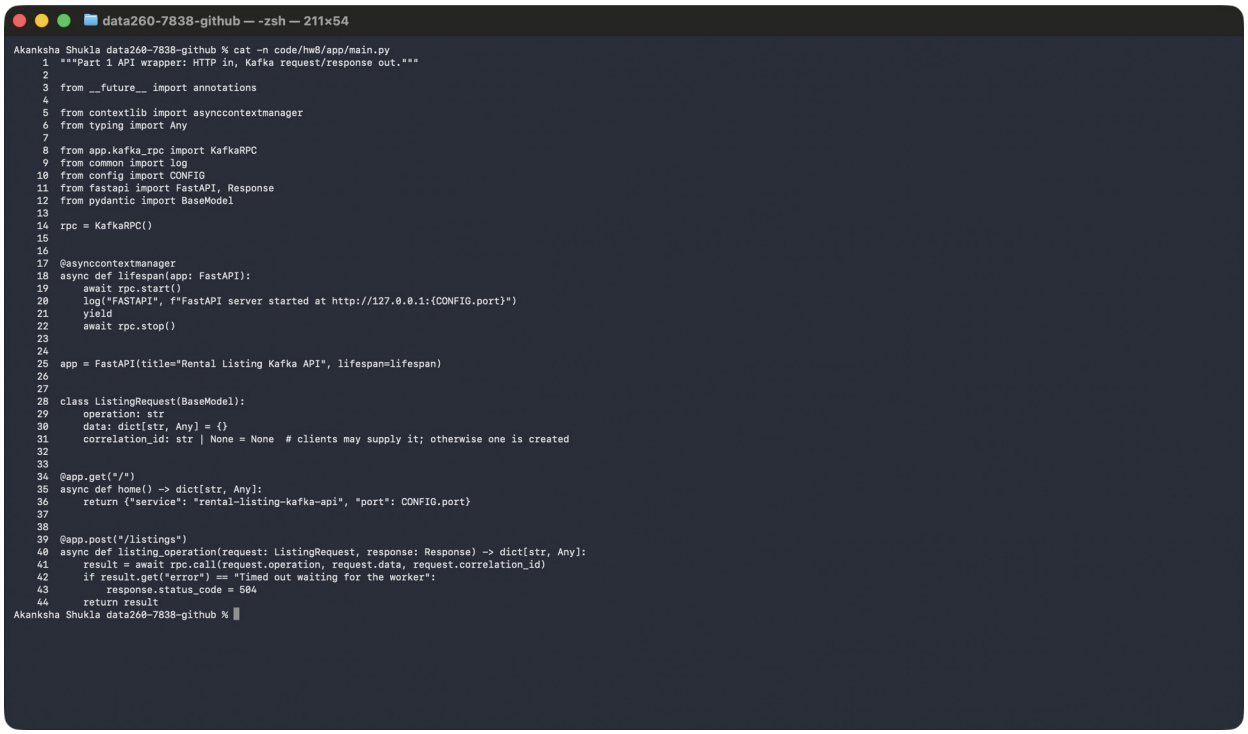
The FastAPI wrapper and Kafka client

- **KafkaRPC.call** builds the request, saves a 'future' (a promise of a future answer) under the correlation_id, publishes the request, and waits up to 10 seconds.
- A background task reads the response topic. When a reply arrives, it looks up the promise by correlation_id and fills it in.
- The wrapper logs 'sending request ...' BEFORE it publishes. The worker can answer so fast that logging afterwards would show the worker's lines out of order.

Screenshot - Kafka client (app/kafka_rpc.py)



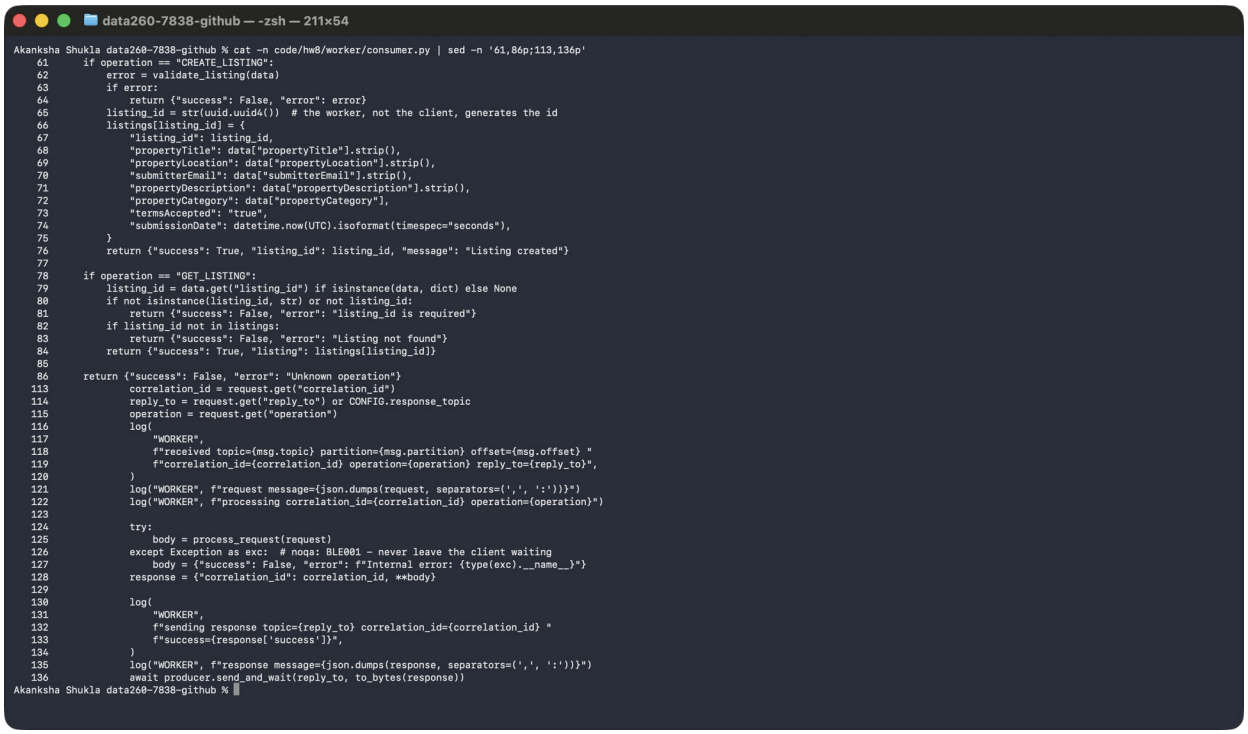
Screenshot - FastAPI wrapper (app/main.py)



The worker

- Reads messages from s7838_entity_requests and checks the operation.
- **CREATE_LISTING:** validates the fields, makes a new listing_id with uuid4 (the worker makes the id, not the client), stores the listing in a plain dictionary, and replies 'Listing created'.
- **GET_LISTING:** looks the id up in the dictionary. If it is missing, it replies success false with 'Listing not found'.
- **Anything else:** replies success false with 'Unknown operation'.
- **Validation rules:** title, location, email and description must be non-empty text; the email must look like an email; the description must be longer than 25 characters after trimming spaces; the category must be one of Apartment, House, Condo, Townhouse; terms must be accepted.
- The worker commits its Kafka offset only after the reply has been sent.

Screenshot - Worker (worker/consumer.py)



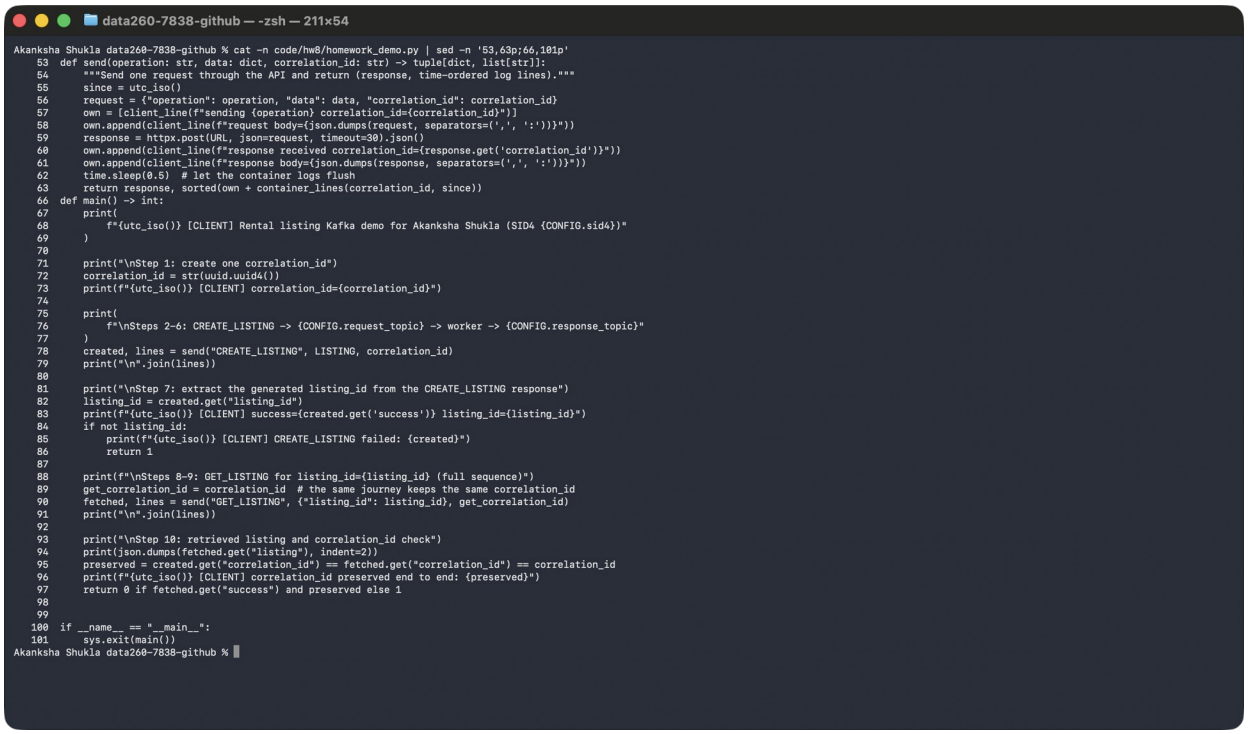
```
data260-7838-github — -zsh — 211×54

Akanksha Shukla data260-7838-github % cat -n code/hw8/worker/consumer.py | sed -n '61,86p;113,136p'
    61      if operation == "CREATE_LISTING":
    62          error = validate_listing(data)
    63          if error:
    64              return {"success": False, "error": error}
    65          listing_id = str(uuid.uuid4())  # the worker, not the client, generates the id
    66          listings[listing_id] = {
    67              "listing_id": listing_id,
    68              "propertyTitle": data["propertyTitle"].strip(),
    69              "propertyLocation": data["propertyLocation"].strip(),
    70              "submitterEmail": data["submitterEmail"].strip(),
    71              "propertyDescription": data["propertyDescription"].strip(),
    72              "propertyCategory": data["propertyCategory"],
    73              "termsAccepted": "true",
    74              "submissionDate": datetime.now(UTC).isoformat(timespec="seconds"),
    75          }
    76          return {"success": True, "listing_id": listing_id, "message": "Listing created"}
    77
    78      if operation == "GET_LISTING":
    79          listing_id = data.get("listing_id") if isinstance(data, dict) else None
    80          if not isinstance(listing_id, str) or not listing_id:
    81              return {"success": False, "error": "listing_id is required"}
    82          if listing_id not in listings:
    83              return {"success": False, "error": "Listing not found"}
    84          return {"success": True, "listing": listings[listing_id]}
    85
    86      return {"success": False, "error": "Unknown operation"}
   113              correlation_id = request.get("correlation_id")
   114              reply_to = request.get("reply_to") or CONFIG.response_topic
   115              operation = request.get("operation")
   116              log(
   117                  "WORKER",
   118                  f"received topic={msg.topic} partition={msg.partition} offset={msg.offset} "
   119                  f"correlation_id={correlation_id} operation={operation} reply_to={reply_to}",
   120              )
   121              log("WORKER", f"request message={json.dumps(request, separators=(',', ':'))}")
   122              log("WORKER", f"processing correlation_id={correlation_id} operation={operation}")
   123
   124              try:
   125                  body = process_request(request)
   126              except Exception as exc:  # noqa: BLE001 — never leave the client waiting
   127                  body = {"success": False, "error": f"Internal error: {type(exc).__name__}"}
   128              response = {"correlation_id": correlation_id, **body}
   129
   130              log(
   131                  "WORKER",
   132                  f"sending response topic={reply_to} correlation_id={correlation_id} "
   133                  f"success={response['success']}",
   134              )
   135              log("WORKER", f"response message={json.dumps(response, separators=(',', ':'))}")
   136              await producer.send_and_wait(reply_to, to_bytes(response))
Akanksha Shukla data260-7838-github %
```

homework_demo.py

- Plays the client and follows the assignment's 10 steps in order: create one correlation_id, send CREATE_LISTING, log the send / worker receive / worker process / worker reply / client response, extract the listing_id, send GET_LISTING, log that whole sequence, display the listing, and confirm the correlation_id is preserved.
- It merges its own log lines with the API and worker container logs (matched by correlation_id) and sorts them by time, so you can read the journey top to bottom.

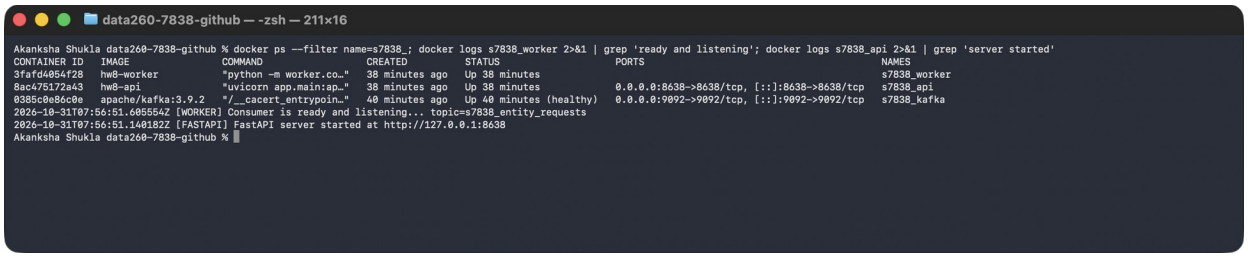
Screenshot - homework_demo.py



Evidence for Part 1

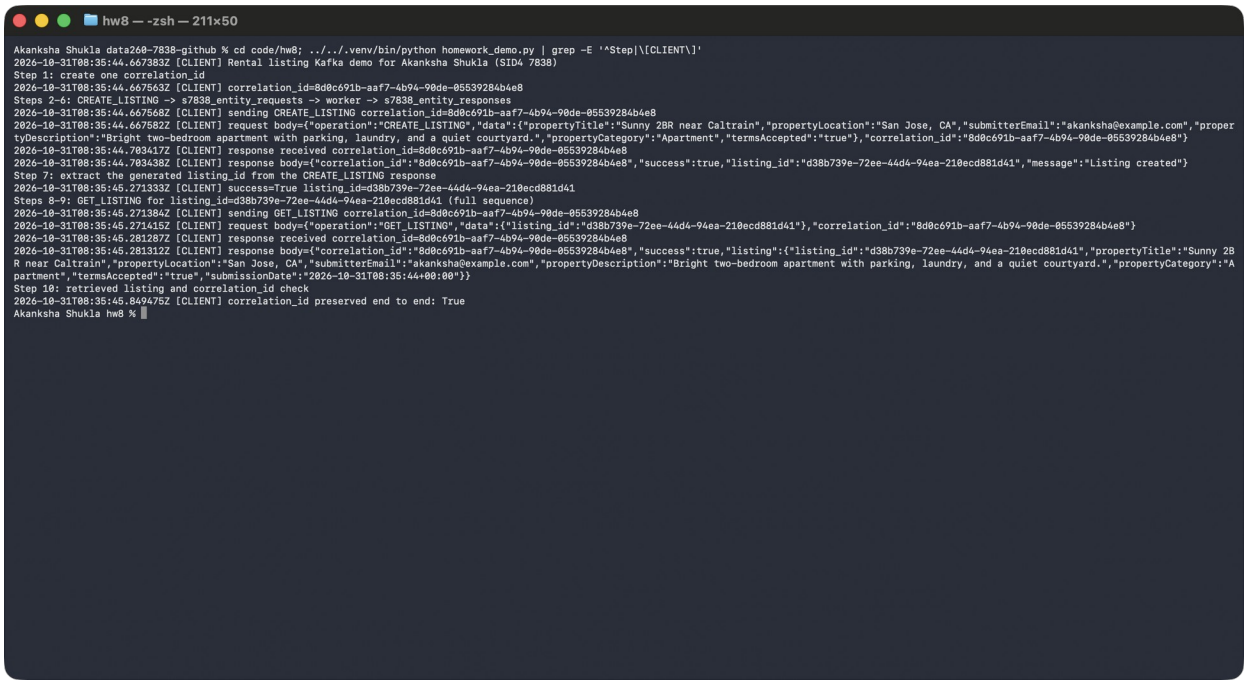
- **Screenshot 1:** shows the three containers running (Kafka, worker, FastAPI), the worker log 'Consumer is ready and listening...' and the FastAPI log 'FastAPI server started at http://127.0.0.1:8638'.

Screenshot - docker ps and the worker/API ready logs



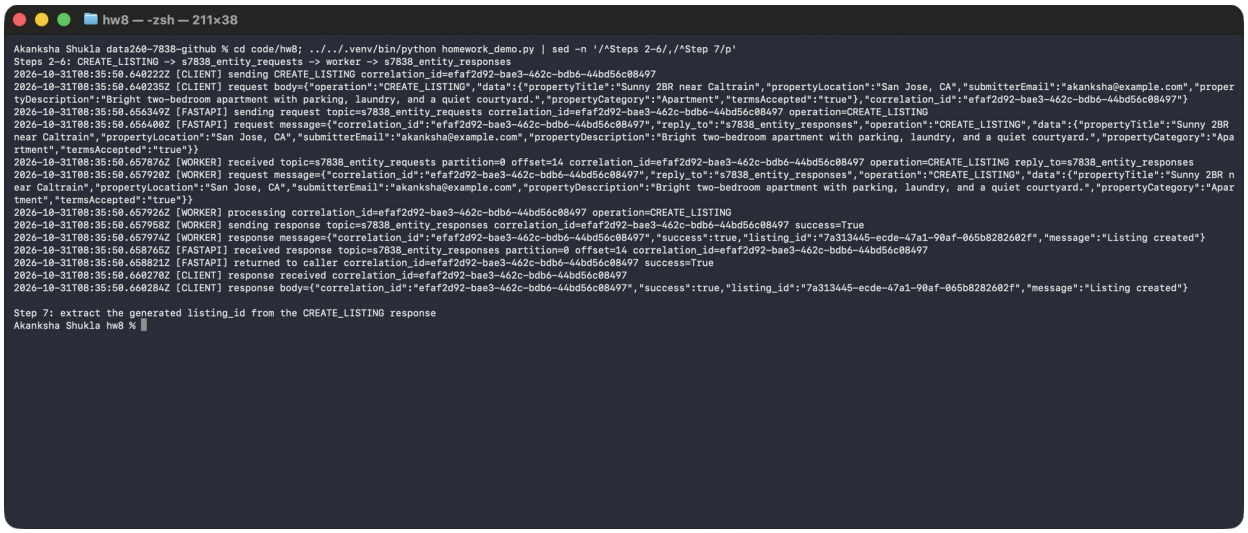
- **Screenshot 2:** shows both operations. Look for: the operation name, the listing data, the generated listing_id, success true, and the same correlation_id in the request and the reply.

Screenshot - CREATE_LISTING and GET_LISTING execution



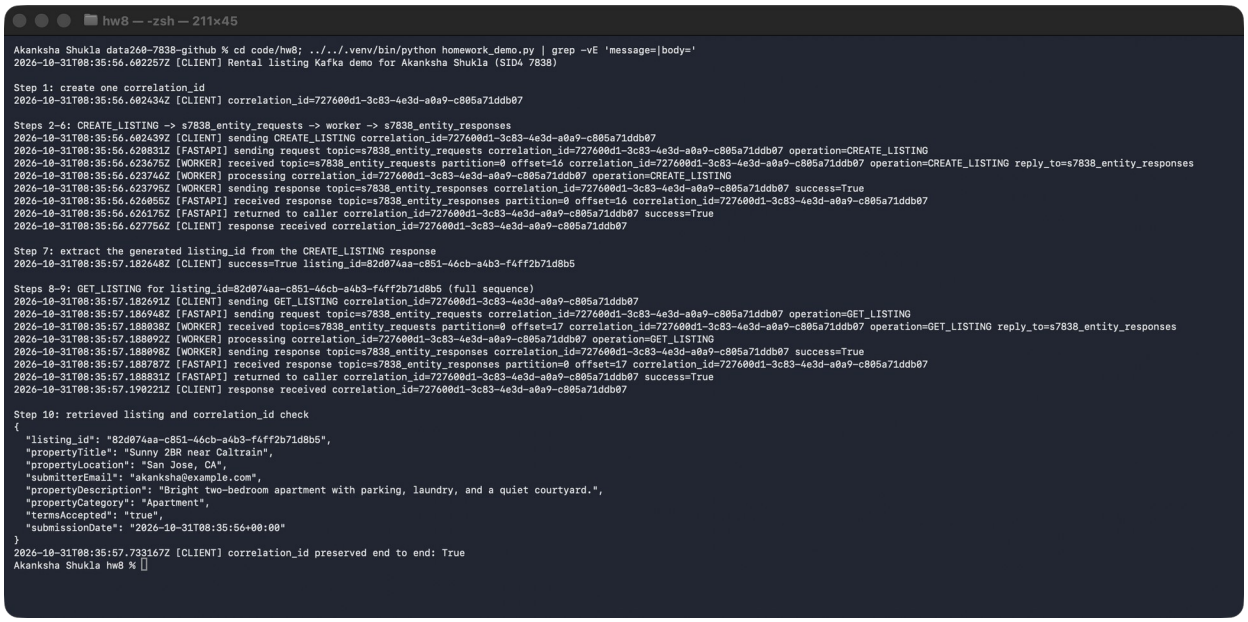
- **Screenshot 3:** shows one CREATE_LISTING travelling through Kafka. The lines are in order: client sends, FastAPI publishes to the request topic, worker receives (with partition and offset), worker processes, worker sends the reply to the response topic, FastAPI receives it, FastAPI returns it to the caller, client gets the response.

Screenshot - Kafka message flow for CREATE_LISTING



- **Screenshot 4:** the same story for CREATE_LISTING followed by GET_LISTING, with the long JSON lines hidden so the order is easy to see. The final lines print the retrieved listing and confirm the correlation_id.

Screenshot - Full CREATE -> GET sequence in order



Part 2 - Three-agent Kafka pipeline

What it does, in plain words

- Three small programs (agents) work together like an assembly line: the **Planner** makes a plan, the **Writer** writes a draft answer from the plan, and the **Reviewer** checks the draft and approves it.
- No agent calls another agent. Each one only reads one topic and writes to the next topic. Kafka sits between them.
- Each agent runs as its own separate process, in its own terminal.

Sender	Reads topic	Sends message to
I (send_question.py)	-	s7838_inbox (a question)
Planner (planner.py)	s7838_inbox	s7838_tasks (a plan)
Writer (writer.py)	s7838_tasks	s7838_drafts (a draft answer)
Reviewer (reviewer.py)	s7838_drafts	s7838_final (an approval)

- **s7838_tasks has 2 partitions;** the other topics have 1. The 2 partitions let two Writers share the work in Part 3.
- All four topics, plus the two from Part 1 and the dead-letter topic, were created with the Kafka command-line tool (kafka-topics.sh) by a one-time job in Docker Compose.
- The Docker Compose file also starts Kafka, the worker and FastAPI. The agents run on my laptop in terminals, using the local qwen2:7b model through Ollama.

What each agent does

- **Planner:** reads the question and asks qwen2:7b for exactly three short numbered steps. It keeps only the numbered lines. If the model returns fewer than two steps, it raises an error (which the retry rules in Part 3 handle).
- **Writer:** reads the question and plan, then asks qwen2:7b to write a 60-100 word answer that follows the plan.

- **Reviewer:** approves the draft if it has at least 15 words. This is a fixed rule, not an AI opinion, so the result is predictable. The final message is JSON with a status and an answer.
- Every agent copies the correlation_id from the message it received into the message it sends.

```
# code/hw8/planner.py
async def handle(message: dict[str, Any]) -> dict[str, Any]:
    question = message.get("question")
    if not isinstance(question, str) or not question.strip():
        raise ValueError("inbox message needs a non-empty question")
    steps = await plan_steps(question)
    if len(steps) < 2:
        raise ValueError(f"model returned fewer than two plan steps: {steps}")
    result = {
        "correlation_id": message["correlation_id"],  # preserved unchanged
        "question": question,
        "plan": steps[:3],
    }
    if message.get("entity_id"):
        result["entity_id"] = message["entity_id"]
    return result
```

```
# code/hw8/writer.py
async def handle(message: dict[str, Any]) -> dict[str, Any]:
    question, plan = message.get("question"), message.get("plan")
    if not isinstance(question, str) or not question.strip():
        raise ValueError("task message needs a non-empty question")
    if not isinstance(plan, list) or not plan:
        raise ValueError("task message needs a non-empty plan list")
    # work_ms lets the failure tests make processing take a known amount of time.
    await asyncio.sleep(float(message.get("work_ms", 0)) / 1000)
    result = {
        "correlation_id": message["correlation_id"],  # preserved unchanged
        "question": question,
        "plan": plan,
        "draft": await write_draft(question, plan),
    }
    for field in ("entity_id", "seq"):
        if field in message:
            result[field] = message[field]
    return result
```

```
# code/hw8/reviewer.py
async def handle(message: dict[str, Any]) -> dict[str, Any]:
    draft = message.get("draft")
    if not isinstance(draft, str):
        raise ValueError("draft message needs a draft string")
    # Objective review rule: a usable answer is non-empty prose with at least MIN_WORDS words.
    approved = len(draft.split()) >= MIN_WORDS
    return {
        "correlation_id": message["correlation_id"],  # preserved unchanged
        "status": "approved" if approved else "rejected",
        "answer": draft,
    }
```

- The model call uses temperature 0 and seed 7838, which keeps answers as repeatable as possible.

How to run it (four terminals)

```
Terminal 1:  python planner.py
Terminal 2:  python writer.py
Terminal 3:  python reviewer.py
Terminal 4:  python send_question.py
```

Evidence for Part 2

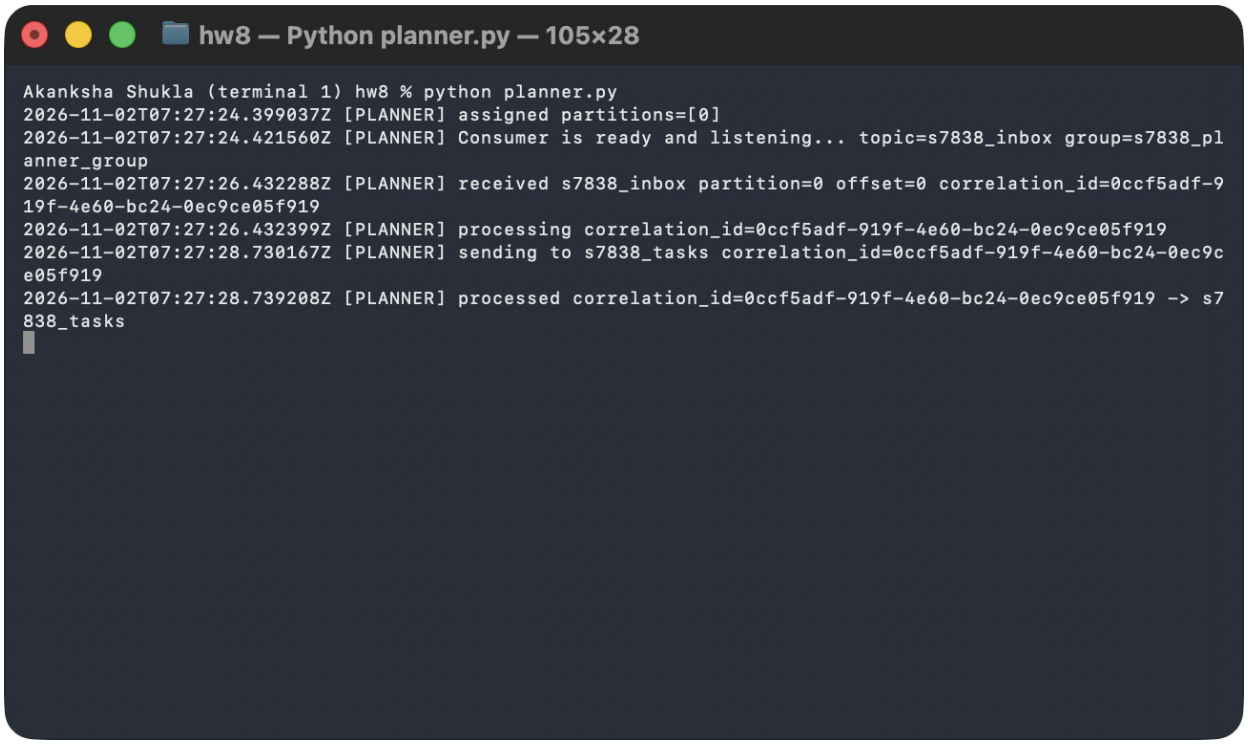
- **Screenshot 5:** the Kafka tool listing all topics. s7838_tasks shows PartitionCount 2; the rest show 1.

Screenshot - Topics created with the Kafka command-line tool

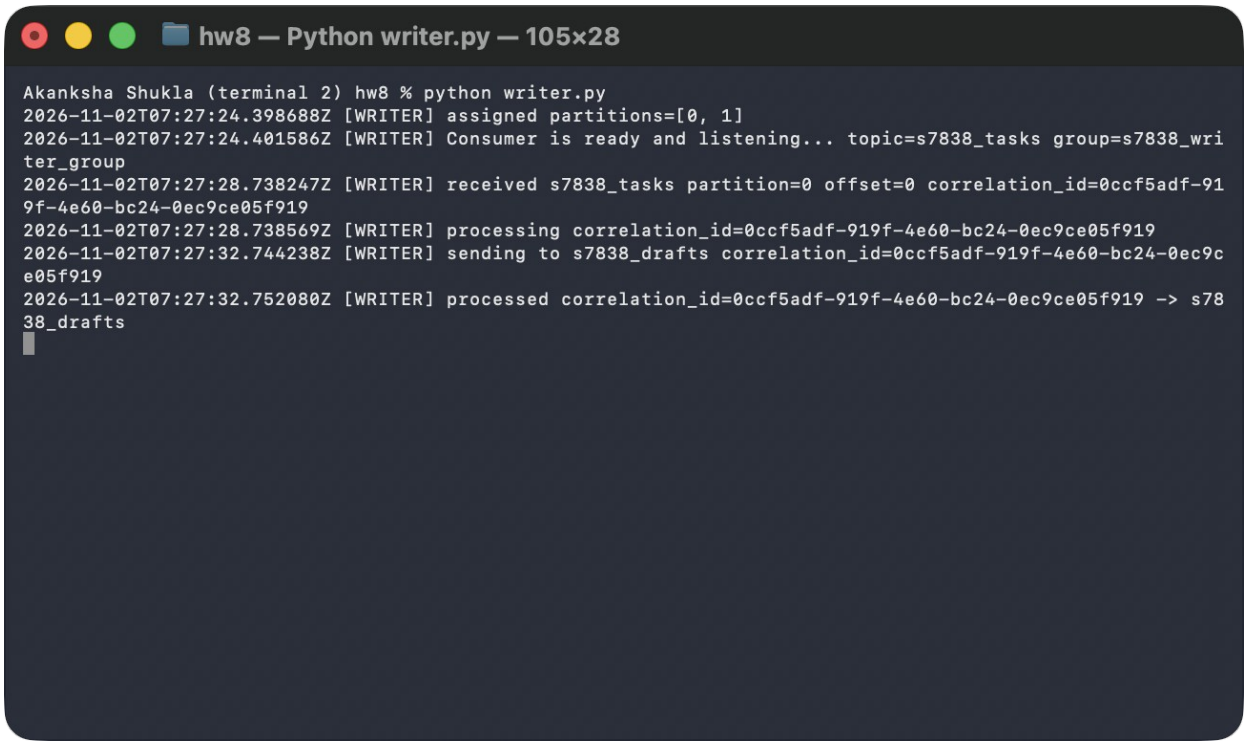


- **Screenshots 6a to 6c (terminals 1 to 3):** the Planner, Writer and Reviewer each running in their own terminal, with log lines for one message (received, processing, sending, processed). The model was qwen2:7b. Every line carries the same correlation_id.

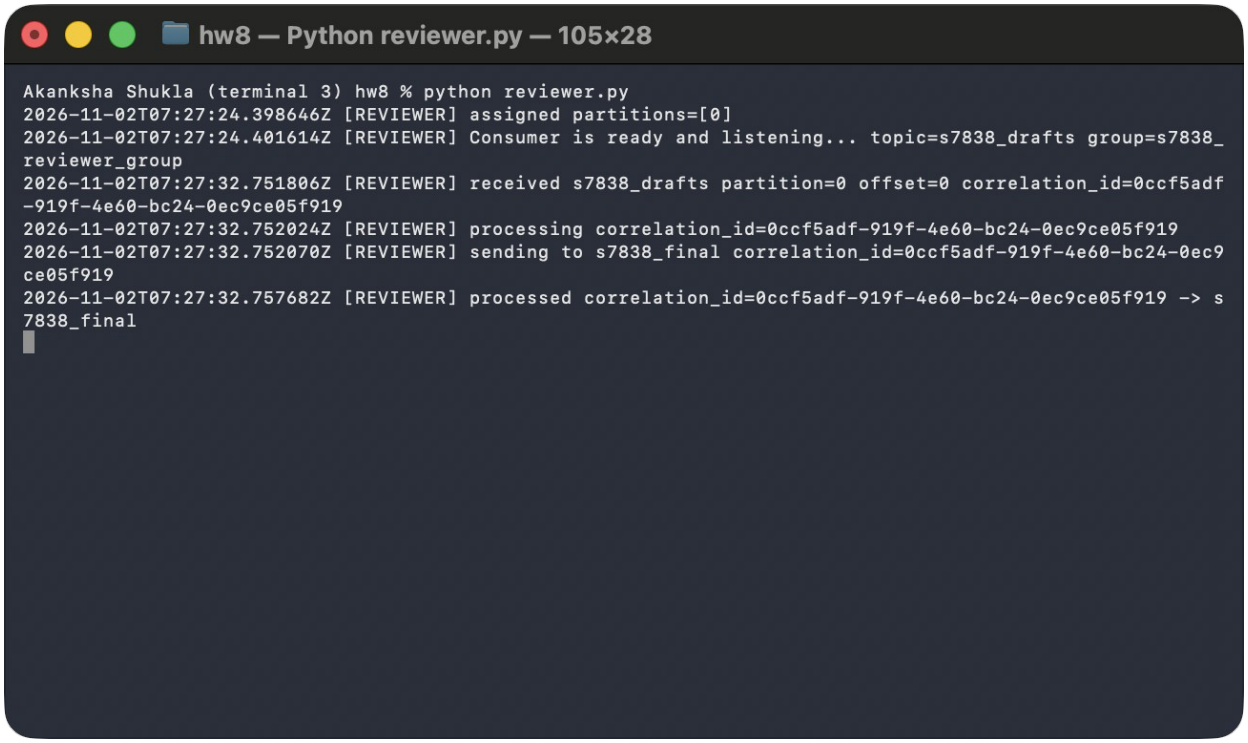
Terminal 1: python planner.py



Terminal 2: python writer.py

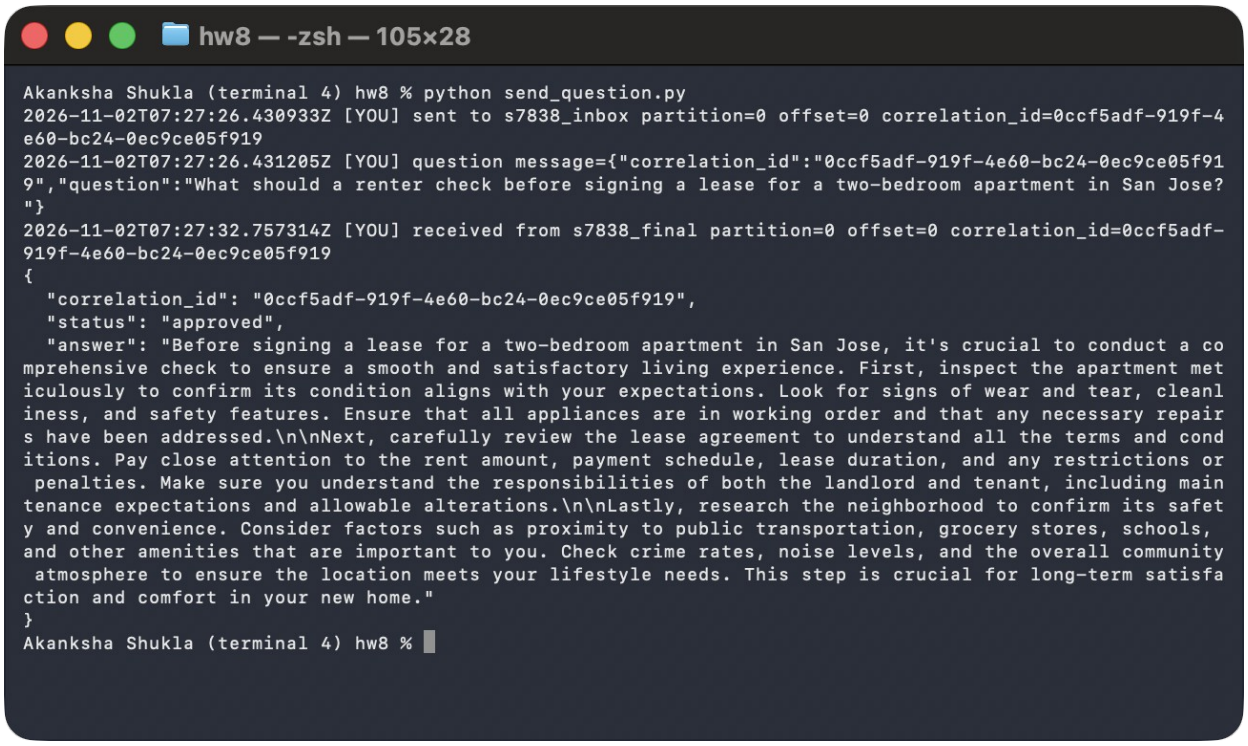


Terminal 3: python reviewer.py



- **Screenshot 7 (terminal 4):** send_question.py sends the question to s7838_inbox and prints the final JSON message read from s7838_final. It has the same correlation_id, a status of approved, and the answer text.

Screenshot - Terminal 4: python send_question.py and the final JSON message



How Kafka helped the agents coordinate

- **No direct calls:** each agent only needs to know its input topic and output topic, so I can change or restart one agent without touching the others.
- **Waiting room:** if an agent is busy (a model call takes several seconds) or restarted, its messages simply wait in the topic. Nothing is lost and the sender is not blocked.
- **One label per question:** the correlation_id travels through all four topics, so the final answer is tied to the original question even with many questions in flight.
- **Easy to inspect:** every hop is a message in a topic with a partition and offset, so I can read the log of a journey line by line.

Part 3 - Failure behavior

Real systems fail in ways a single program hides. I tested four failures. For each one I ran the Writer 'before the fix' (the fix turned off) and 'after the fix' (the fix turned on), against the real Kafka running in Docker. The Writer was a real separate process; to keep each run quick and repeatable it used a simple stand-in for the model. Only one setting changed between the before and after run of each test.

Summary table (before and after)

Failure scenario	Before fix	After fix
Duplicate delivery	Same correlation_id delivered twice: processed 2 times, 2 drafts published (a repeated side effect).	Idempotency: processed 1 time, 1 duplicate skipped, 1 draft published.
Consumer killed mid-work	Offset committed on receipt. After the kill the committed offset was 6 (message was at 5), so the restarted Writer never saw the message. Drafts published: 0. The message was lost.	Offset committed after processing. After the kill it was still 6 (message at 6). The restarted Writer received it again, finished it, and committed 7. Drafts published: 1.

Failure scenario	Before fix	After fix
Poison message	Unlimited retries: 54 failed attempts in 12 seconds, never moved to a dead-letter topic. The next good message was blocked behind it.	3 attempts, then 1 record in s7838_dead_letter. The next good message was processed.
Two partitions / two consumers	No message key: related messages were spread across both partitions. Listing-1 was processed out of order: 1, 2, 5, 3, 4, 6, 8, 7.	Key = entity_id: every listing stayed in one partition and was processed in order.

I. Duplicate delivery

- **What can go wrong:** Kafka promises 'at least once' delivery, so the same message can arrive twice (for example after a retry or a restart). Without protection, the Writer would do the same work twice.
- **Test:** I sent the exact same message (same correlation_id) twice to s7838_tasks.
- **Before the fix:** the Writer processed both copies and published 2 drafts for one question.
- **The fix (idempotency):** the Writer keeps a small SQLite table of correlation_ids it has already finished. Before doing work it checks the table; if the id is there, it logs 'duplicate skipped' and does nothing. After a successful job it saves the id.
- **After the fix:** the second copy was skipped. Processed 1, skipped 1, drafts published 1.

```
# code/hw8/reliability.py
class IdempotencyStore:
    """Durable set of processed correlation_ids (SQLite file, so restarts keep it)."""

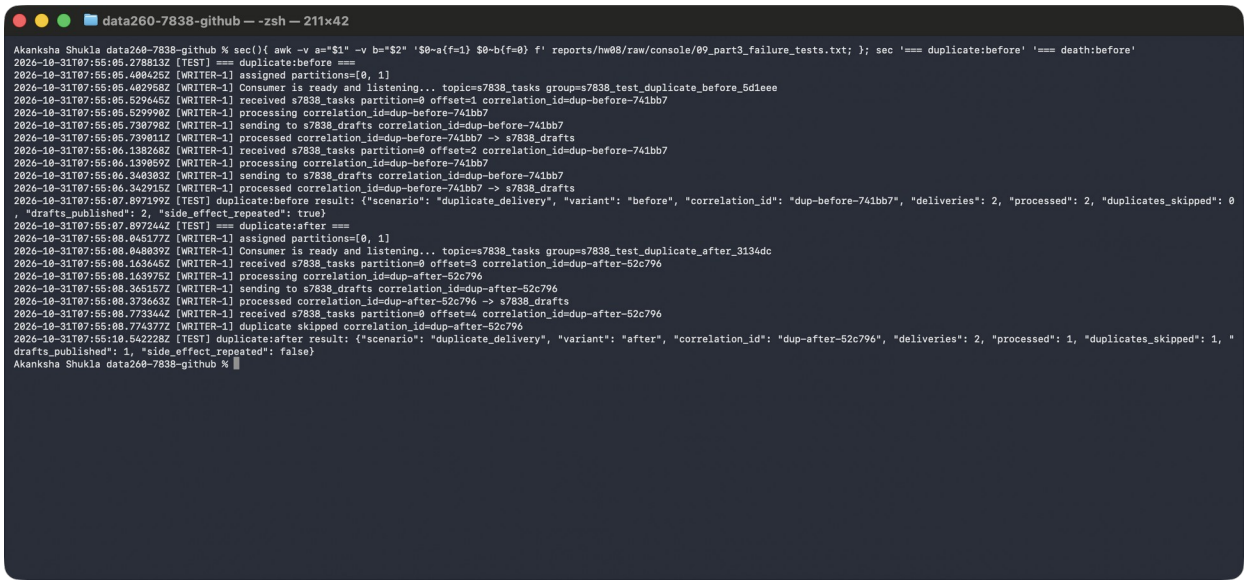
    def __init__(self, path: str | Path) -> None:
        Path(path).parent.mkdir(parents=True, exist_ok=True)
        self.db = sqlite3.connect(path)
        self.db.execute("CREATE TABLE IF NOT EXISTS processed (correlation_id TEXT PRIMARY KEY)")
        self.db.commit()

    def seen(self, correlation_id: str) -> bool:
        row = self.db.execute(
            "SELECT 1 FROM processed WHERE correlation_id = ?", (correlation_id,)
        ).fetchone()
        return row is not None

    def mark(self, correlation_id: str) -> None:
        self.db.execute("INSERT OR IGNORE INTO processed VALUES (?)", (correlation_id,))
        self.db.commit()
```

```
# code/hw8/agent_runtime.py
            if store and correlation_id and store.seen(correlation_id):
                log(s.label, f"duplicate skipped correlation_id={correlation_id}")
                emit("duplicate_skipped", **detail)
```

Screenshot - Replay of the same correlation_id, before and after idempotency



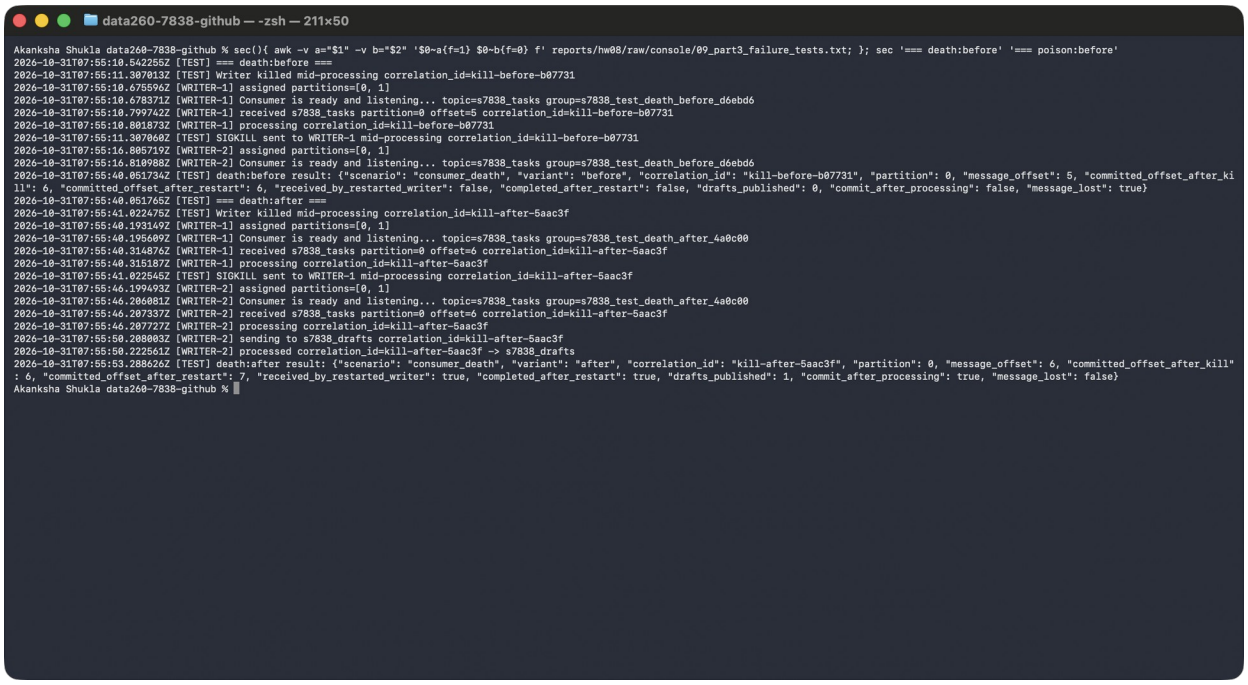
II. Consumer death

- **What can go wrong:** a consumer can crash in the middle of a job. If it already told Kafka 'done' (committed the offset) before finishing, Kafka will never send that message again and the work is lost.
- **Test:** I sent a message that takes 4 seconds to process, then killed the Writer with SIGKILL while it was working, then started a new Writer in the same consumer group.
- **Before the fix (commit on receipt):** the offset was committed as soon as the message arrived. After the kill the committed offset was 6, which is past the message (offset 5). The restarted Writer never saw it and no draft was published.
- **The fix (commit after processing):** the Writer commits only after the work is finished and the output is published.
- **After the fix:** after the kill the committed offset was still 6, which is the message's own offset (6), so Kafka had not forgotten it. The restarted Writer received it again, finished it, published 1 draft, and only then committed 7.

```
# code/hw8/reliability.py
async def commit_offset(consumer: AIOKafkaConsumer, msg: Any) -> None:
    """Commit "next message to read" for this message's partition."""
    await consumer.commit({TopicPartition(msg.topic, msg.partition): msg.offset + 1})

# code/hw8/agent_runtime.py
            if s.commit_mode == "early":  # "before": commit on receipt, work not done yet
                await commit_offset(consumer, msg)
                emit("committed", **detail)
    ...
            if s.commit_mode == "late":  # "after": commit only once the work is done
                await commit_offset(consumer, msg)
                emit("committed", **detail)
```

Screenshot - Writer killed mid-processing, then restarted



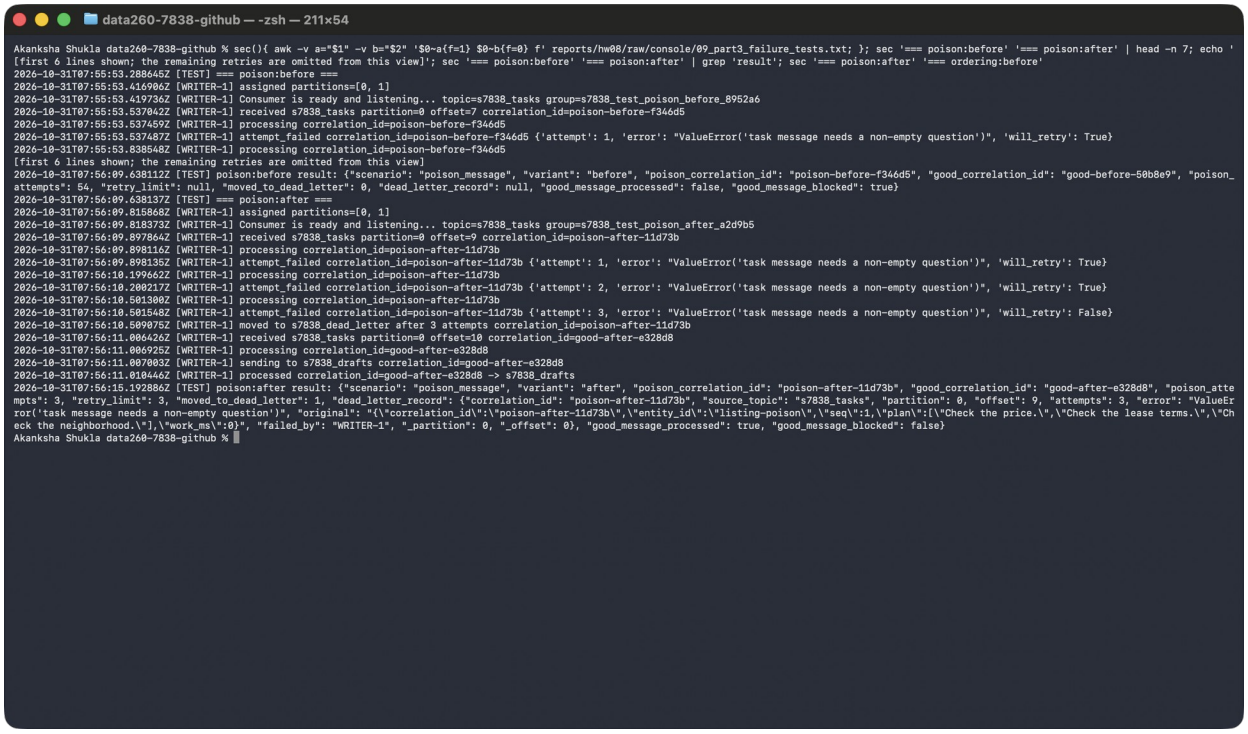
III. Poison message

- **What can go wrong:** a message that always fails (a 'poison message') can block everything behind it if the consumer keeps retrying forever.
- **Test:** I sent a task with no question (the Writer cannot handle that), followed by a normal task with the same key, so they were in the same partition. The poison message came first.
- **Before the fix (unlimited retries):** the Writer retried 54 times in 12 seconds and never gave up. The good message behind it was never processed.
- **The fix:** bounded retries. The Writer tries up to 3 times, then sends the bad message to the dead-letter topic s7838_dead_letter (with the error, the attempt count and the original text), commits it, and moves on.
- **After the fix:** 3 attempts, 1 record in the dead-letter topic, and the good message was processed right after.

```
# code/hw8/reliability.py
async def run_with_retries(
    attempt_once: Callable[[], Awaitable[Any]],
    retry_mode: str,
    max_attempts: int,
    emit: Callable[..., None],
    backoff: float = 0.3,
) -> tuple[bool, Any, int]:
    """Return (ok, output_or_error, attempts).

    bounded:   stop after max_attempts so the caller can dead-letter the message.
    unbounded: never gives up (the "before" behavior that blocks the partition).
    """
    attempt = 0
    while True:
        attempt += 1
        try:
            return True, await attempt_once(), attempt
        except Exception as exc:  # noqa: BLE001 - any failure counts as a failed attempt
            will_retry = retry_mode != "bounded" or attempt < max_attempts
            emit("attempt_failed", attempt=attempt, error=repr(exc)[:160], will_retry=will_retry)
            if not will_retry:
                return False, repr(exc)[:160], attempt
            await asyncio.sleep(backoff)
```

Screenshot - Poison message: unlimited retries versus bounded retries and dead letter

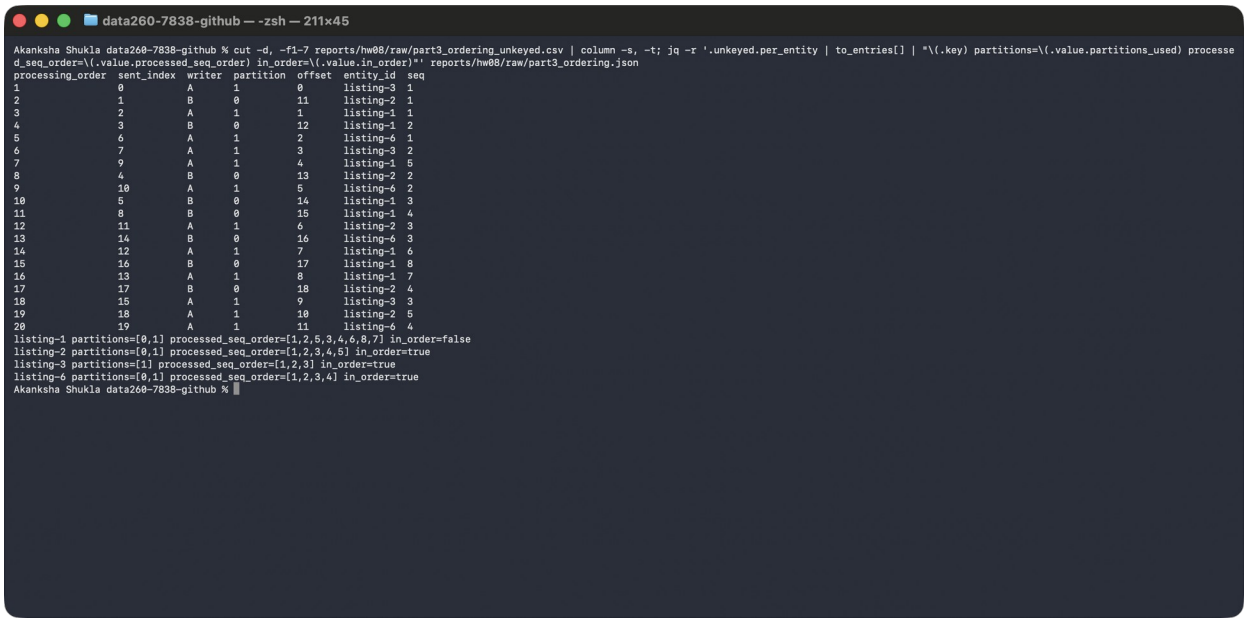


IV. Ordering

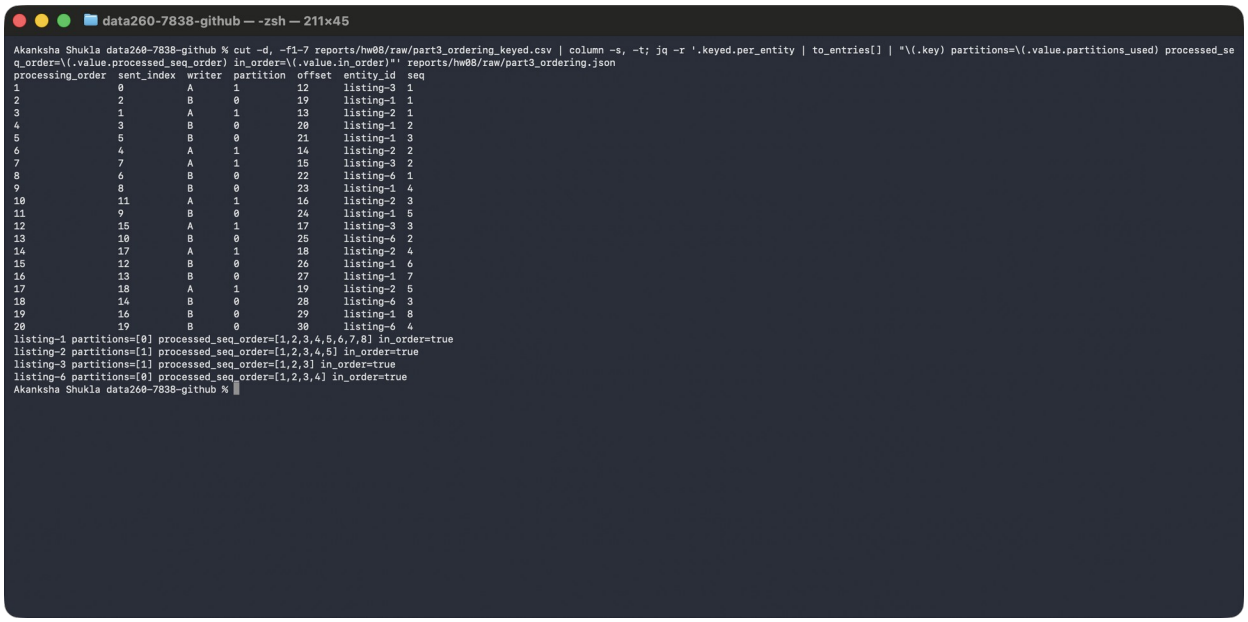
- **What can go wrong:** Kafka keeps order only inside one partition. s7838_tasks has 2 partitions, and two Writers work in parallel. If messages about the same listing land in different partitions, they can finish out of order.
- **Test:** using VERIFY_SEED 267838, I generated 20 related messages about 4 listings (listing-1, 2, 3 and 6), each with a sequence number. I sent them through s7838_tasks with two Writers (A and B) in one consumer group and recorded the order they were processed.
- **Before the fix (no key):** related messages were spread across both partitions. Listing-1 was processed in the order 1, 2, 5, 3, 4, 6, 8, 7 instead of 1 to 8. Listings listing-1, listing-2, listing-6 were split across both partitions.
- **The fix (partition key):** I use the entity (listing id) as the message key. Kafka sends all messages with the same key to the same partition, so one Writer handles them in order.
- **After the fix:** every listing stayed in one partition and was processed in order (listing-1: 1 to 8 in order).
- The unkeyed run assigns partitions with a seeded random generator, standing in for a producer that sends without keys. The raw rows are saved in raw/part3_ordering_unkeyed.csv and raw/part3_ordering_keyed.csv.

```
# code/hw8/reliability.py
def partition_key(message: dict[str, Any]) -> bytes:
    """Key by the entity the messages are about (falling back to the journey's correlation_id)."""
    return str(message.get("entity_id") or message["correlation_id"]).encode()
```

Screenshot - Without a key: related messages are processed out of order



Screenshot - With key = entity_id: related messages stay in one partition, in order



Comparison with the HW2 in-process graph

In HW2 my Planner/Reviewer graph ran as function calls inside one program: each step waited for the previous one directly. With Kafka the steps are separate programs connected by messages.

Two benefits I gained:

- **Independent scaling:** I can start more Writers in the same consumer group (my ordering test used two) while Planner and Reviewer stay at one. In HW2 the only way to handle more work was to copy the whole program.
- **Decoupled failure:** when the Writer was killed, its message stayed safely in Kafka and was finished after the restart. In HW2 a failure in one step failed the whole run, with nothing saved to resume from.

Two responsibilities I now handle myself:

- **Duplicates and bad messages:** Kafka delivers at least once, so I had to add idempotency, a retry limit, and a dead-letter topic. A function call in HW2 ran exactly once per step.
- **Ordering and offsets:** order only holds inside a partition, so I had to choose a partition key, and I have to commit offsets at the right moment (after the work). In HW2 the order of the calls was the order of the work.

Self-check (verification.json)

- A script (make verify-hw08) starts the system and runs a quick smoke test. A smoke test only checks that the basics work, not every edge case. It writes verification.json.
- It records the homework number (8), SID4 (7838), the commit hash, the model and settings, SEED and VERIFY_SEED, each check, and pass/fail for each check.
- The checks test behavior, not exact wording. The 12 checks are listed below.

    PASS - All seven PREFIX topics exist (tasks has 2 partitions)

    PASS - CREATE_LISTING returns success, a UUID listing_id, and the same correlation_id

    PASS - GET_LISTING returns the created listing with the same correlation_id

    PASS - Unknown listing_id gives success=false, an error, and the same correlation_id

    PASS - Unknown operation is rejected with a clean error and the same correlation_id

    PASS - Missing, empty, and invalid fields are rejected

    PASS - Worker container logged 'Consumer is ready and listening...'

    PASS - Pipeline returns an approved answer on s7838_final with the same correlation_id

    PASS - Replayed message causes no extra side effect (idempotency)

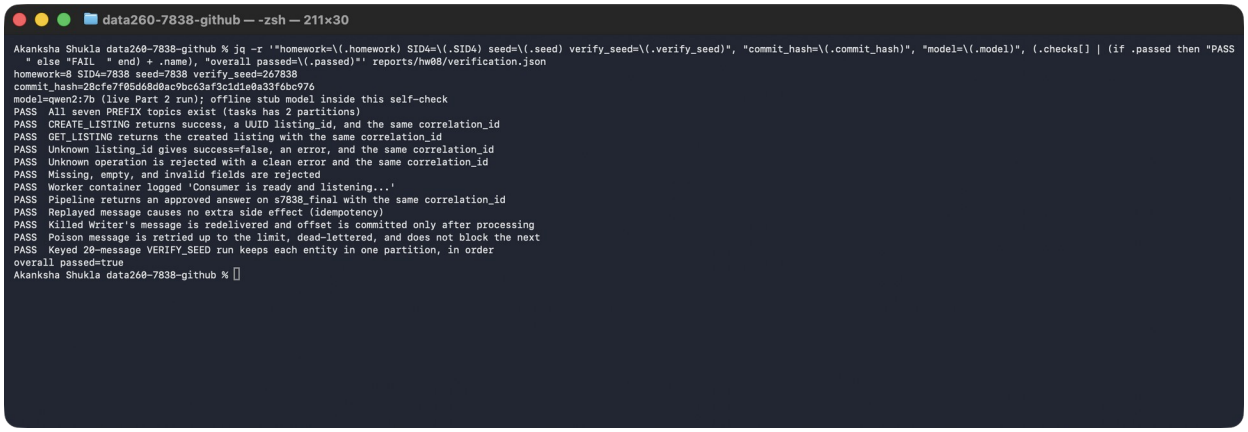
    PASS - Killed Writer's message is redelivered and offset is committed only after processing

    PASS - Poison message is retried up to the limit, dead-lettered, and does not block the next

    PASS - Keyed 20-message VERIFY_SEED run keeps each entity in one partition, in order

- The script only creates temporary files and writes verification.json; it does not change any application code.
- Its pipeline and failure checks use an offline stand-in model so the check is quick; the live Part 2 run above used qwen2:7b.

Screenshot - make verify-hw08 wrote verification.json (all checks pass)



AI use

1. What I used an AI assistant for, and what I did myself

I used Claude Code to verify that I have not missed any important part of the HW8 requirements for eg - Kafka worker, FastAPI wrapper, three agents, Docker Compose file, failure-test harness, self-check script, and report generator, following the structure of my earlier homework folders.

2. One AI output that was wrong or unsuitable

The first FastAPI wrapper created its Kafka producer and consumer when the module was imported, a pattern copied from the example project. With the installed aiokafka 0.15 this was wrong: the API container crashed at startup with "The object should be created within an async function" and kept restarting. A second problem appeared in the first demo run: merging client, API, and worker log lines by millisecond timestamp put the worker's "processing" line before its "received" line, which would have misdescribed the message flow.

3. How I detected the problem or verified the result

I found the crash by running `docker ps` (the API container showed a fresh restart each time) and reading `docker logs s7838_api`, and I found the log-order problem by reading the first `homework_demo.py` output against the expected order of the ten required steps. I then verified the fix with a real CREATE_LISTING followed by GET_LISTING through Kafka, and with the self-check script, which tests that the same correlation_id comes back on every response.

4. What I changed and why it works now

I moved client creation into `KafkaRPC.start()`, which runs inside the event loop, and I rebuilt the image. I switched log timestamps to microseconds and made the API and worker log "sending ..." before they publish, so each line shows the order in which things happened. The API now starts cleanly, logs `FastAPI server started at http://127.0.0.1:8638`, and the demo prints send, receive, process, reply, and client-response lines in the correct order.

Where everything is in the repository

- **code/hw8/:** all HW8 code (worker, FastAPI app, agents, reliability fixes, demo, tests, Docker Compose file).
- **reports/hw08/RUN_LOG.txt:** the real console output and timestamps of the full run.
- **reports/hw08/raw/:** the saved CSV/JSON data, including the 20-message ordering run and the Part 3 event logs.
- **reports/hw08/METRICS.md:** the results tables.
- **reports/hw08/AI_USE.md:** the AI-use answers.
- **reports/hw08/verification.json:** the self-check result.